

# Code Wali PDF: Workbook Generator

This PDF contains the Python code used to generate the workbook PDF and its 10 original images.

```

0001: from pathlib import Path
0002: from textwrap import wrap
0003: import math
0004:
0005: from PIL import Image, ImageDraw, ImageFont
0006: from reportlab.lib import colors
0007: from reportlab.lib.pagesizes import A4
0008: from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
0009: from reportlab.lib.units import inch
0010: from reportlab.pdfbase.pdfmetrics import stringWidth
0011: from reportlab.platypus import (
0012:     Image as RLIImage,
0013:     PageBreak,
0014:     Paragraph,
0015:     SimpleDocTemplate,
0016:     Spacer,
0017:     Table,
0018:     TableStyle,
0019: )
0020:
0021:
0022: BASE_DIR = Path(__file__).resolve().parent
0023: ASSET_DIR = BASE_DIR / "generated_assets"
0024: WORKBOOK_PDF = BASE_DIR / "BAPS-IPDC-WORKBOOK-ITMBU-ED-1-VAISVIK-EKTA-SAMP-SEVA.pdf"
0025: CODE_PDF = BASE_DIR / "CODE-VALI-PDF-BAPS-IPDC-WORKBOOK-GENERATOR.pdf"
0026:
0027: PAGE_SIZE = A4
0028: PAGE_WIDTH, PAGE_HEIGHT = PAGE_SIZE
0029:
0030:
0031: THEME = {
0032:     "navy": colors.HexColor("#12324A"),
0033:     "saffron": colors.HexColor("#E46D2E"),
0034:     "green": colors.HexColor("#2D7D5F"),
0035:     "gold": colors.HexColor("#F1B54A"),
0036:     "cream": colors.HexColor("#FFF8EA"),
0037:     "light": colors.HexColor("#F4F7F8"),
0038:     "ink": colors.HexColor("#1F2A33"),
0039:     "muted": colors.HexColor("#5C6670"),
0040: }
0041:
0042:
0043: IMAGE_SPECS = [
0044:     (
0045:         "01_global_circle.png",
0046:         "Global unity circle",
0047:         "Students from many cultures standing around the earth.",
0048:     ),
0049:     (
0050:         "02_samp_bridge.png",
0051:         "Samp bridge",
0052:         "A bridge built from listening, respect, forgiveness, and teamwork.",
0053:     ),
0054:     (
0055:         "03_seva_hands.png",
0056:         "Seva hands",
0057:         "Hands serving food, books, water, and care.",
0058:     ),
0059:     (
0060:         "04_value_tree.png",
0061:         "Value tree",
0062:         "Roots of humility and branches of unity, harmony, and service.",
0063:     ),
0064:     (
0065:         "05_classroom_circle.png",
0066:         "Classroom circle",
0067:         "A classroom reflection circle for peaceful discussion.",
0068:     ),
0069:     (
0070:         "06_service_map.png",
0071:         "Service map",
0072:         "A local to global seva map.",
0073:     ),
0074:     (
0075:         "07_conflict_to_samp.png",
0076:         "Conflict to samp",
0077:         "A simple pathway from ego to understanding.",
0078:     ),
0079:     (
0080:         "08_daily_practice.png",
0081:         "Daily practice wheel",
0082:         "Small daily habits for ekta, samp, and seva.",
0083:     ),
0084:     (
0085:         "09_team_project.png",
0086:         "Team project",
0087:         "Students planning a seva project together.",
0088:     ),
0089:     (
0090:         "10_final_pledge.png",
0091:         "Final pledge",
0092:         "A pledge card surrounded by symbols of unity and service.",
0093:     ),
0094: ]
0095:
0096:
0097: def get_font(size=28, bold=False):
0098:     font_candidates = [

```

```

0099:         "C:/Windows/Fonts/arialbd.ttf" if bold else "C:/Windows/Fonts/arial.ttf",
0100:         "C:/Windows/Fonts/calibrib.ttf" if bold else "C:/Windows/Fonts/calibri.ttf",
0101:     ]
0102:     for candidate in font_candidates:
0103:         if Path(candidate).exists():
0104:             return ImageFont.truetype(candidate, size)
0105:     return ImageFont.load_default()
0106:
0107:
0108: def draw_wrapped(draw, text, box, font, fill, line_spacing=8, align="center"):
0109:     x1, y1, x2, y2 = box
0110:     max_width = x2 - x1
0111:     words = text.split()
0112:     lines = []
0113:     line = ""
0114:     for word in words:
0115:         test = (line + " " + word).strip()
0116:         if draw.textlength(test, font=font) <= max_width:
0117:             line = test
0118:         else:
0119:             if line:
0120:                 lines.append(line)
0121:             line = word
0122:     if line:
0123:         lines.append(line)
0124:     line_height = font.size + line_spacing
0125:     total_height = len(lines) * line_height
0126:     y = y1 + max(0, (y2 - y1 - total_height) / 2)
0127:     for line in lines:
0128:         width = draw.textlength(line, font=font)
0129:         if align == "center":
0130:             x = x1 + (max_width - width) / 2
0131:         elif align == "right":
0132:             x = x2 - width
0133:         else:
0134:             x = x1
0135:         draw.text((x, y), line, fill=fill, font=font)
0136:         y += line_height
0137:
0138:
0139: def rounded_card(draw, xy, fill, outline=None, radius=30, width=4):
0140:     draw.rounded_rectangle(xy, radius=radius, fill=fill, outline=outline, width=width)
0141:
0142:
0143: def save_image(name, title, subtitle, theme_index):
0144:     ASSET_DIR.mkdir(exist_ok=True)
0145:     path = ASSET_DIR / name
0146:     img = Image.new("RGB", (1200, 760), "#FFF8EA")
0147:     d = ImageDraw.Draw(img)
0148:
0149:     palette = [
0150:         ("12324A", "#E46D2E", "#2D7D5F", "#F1B54A"),
0151:         ("214E77", "#35A79C", "#F2C14E", "#DF6C4F"),
0152:         ("2E4057", "#66A182", "#F6AE2D", "#8D99AE"),
0153:         ("1F363D", "#40798C", "#70A9A1", "#F6D8AE"),
0154:     ][theme_index % 4]
0155:     dark, accent, green, gold = palette
0156:
0157:     d.rectangle((0, 0, 1200, 760), fill="#FFF8EA")
0158:     d.rectangle((0, 0, 1200, 90), fill=dark)
0159:     d.text((55, 22), title, fill="white", font=get_font(38, True))
0160:     d.text((55, 640), subtitle, fill=dark, font=get_font(28, False))
0161:
0162:     if "01_" in name:
0163:         d.ellipse((395, 145, 805, 555), fill="#A9D6E5", outline=dark, width=7)
0164:         d.arc((435, 180, 780, 520), 205, 340, fill=green, width=18)
0165:         d.arc((430, 175, 780, 510), 25, 160, fill=accent, width=18)
0166:         d.arc((500, 220, 715, 480), 100, 280, fill=gold, width=14)
0167:         for x, y, c in [(220, 230, accent), (960, 230, green), (250, 510, gold), (930, 510, "#DF6C4F")]:
0168:             d.ellipse((x - 45, y - 45, x + 45, y + 45), fill=c, outline=dark, width=5)
0169:             d.line((x, y + 45, x, y + 130), fill=dark, width=8)
0170:             d.line((x - 55, y + 90, x + 55, y + 90), fill=dark, width=7)
0171:             d.line((x, y + 130, x - 45, y + 190), fill=dark, width=7)
0172:             d.line((x, y + 130, x + 45, y + 190), fill=dark, width=7)
0173:     elif "02_" in name:
0174:         for i, word in enumerate(["Listening", "Respect", "Forgiveness", "Teamwork"]):
0175:             x = 145 + i * 230
0176:             rounded_card(d, (x, 335, x + 185, 455), fill="FFFFFF", outline=dark, radius=22)
0177:             draw_wrapped(d, word, (x + 14, 355, x + 171, 440), get_font(26, True), dark)
0178:             d.line((150, 455, 1050, 455), fill=dark, width=16)
0179:             d.arc((130, 240, 1070, 690), 190, 350, fill=accent, width=18)
0180:             d.text((460, 175), "SAMP", fill=accent, font=get_font(72, True))
0181:     elif "03_" in name:
0182:         items = [{"Food", 210, 260, accent}, {"Books", 430, 390, green}, {"Water", 650, 260, gold}, {"Care", 870, 390, "#DF6C4F"}]
0183:         for label, x, y, c in items:
0184:             d.rounded_rectangle((x, y, x + 145, y + 105), radius=25, fill=c, outline=dark, width=5)
0185:             draw_wrapped(d, label, (x + 8, y + 28, x + 137, y + 80), get_font(28, True), "white")
0186:             d.line((x + 70, y + 105, 600, 555), fill=dark, width=8)
0187:             d.ellipse((500, 500, 700, 700), fill=green, outline=dark, width=6)
0188:             draw_wrapped(d, "SEVA", (510, 555, 690, 635), get_font(38, True), "white")
0189:     elif "04_" in name:
0190:         d.rectangle((560, 340, 640, 605), fill="#8B5E34")
0191:         d.polygon([(600, 145), (300, 400), (900, 400)], fill=green, outline=dark)
0192:         d.polygon([(600, 225), (380, 470), (820, 470)], fill="#66A182", outline=dark)
0193:         for label, x, y in [{"Humility", 230, 585}, {"Faith", 500, 625}, {"Discipline", 765, 585}]:
0194:             d.line((600, 600, x + 80, y), fill=dark, width=8)
0195:             draw_wrapped(d, label, (x, y, x + 170, y + 55), get_font(24, True), dark)
0196:         for label, x, y in [{"Ekta", 360, 300}, {"Samp", 535, 225}, {"Seva", 710, 300}]:
0197:             draw_wrapped(d, label, (x, y, x + 130, y + 60), get_font(30, True), "white")
0198:     elif "05_" in name:
0199:         center = (600, 385)
0200:         for i, c in enumerate([accent, green, gold, "#DF6C4F", "#6A7FDB", "#35A79C", "#A7754D", "#7BA05B"]):
0201:             angle = math.radians(i * 45)

```

```

0202:         x = center[0] + 280 * math.cos(angle)
0203:         y = center[1] + 190 * math.sin(angle)
0204:         d.ellipse((x - 45, y - 45, x + 45, y + 45), fill=c, outline=dark, width=4)
0205:         d.ellipse((415, 260, 785, 510), fill="white", outline=dark, width=7)
0206:         draw_wrapped(d, "Speak with respect. Listen to understand.", (455, 315, 745, 450), get_font(32,
True), dark)
0207:         elif "06_" in name:
0208:             d.line((190, 545, 1010, 210), fill=dark, width=10)
0209:             points = [{"Home", 170, 520, accent), ("Campus", 390, 430, green), ("City", 610, 340, gold),
("Nation", 830, 250, "#DF6C4F"), ("World", 1010, 175, "#6A7FDB"]}
0210:             for label, x, y, c in points:
0211:                 d.ellipse((x - 58, y - 58, x + 58, y + 58), fill=c, outline=dark, width=5)
0212:                 draw_wrapped(d, label, (x - 52, y - 20, x + 52, y + 25), get_font(24, True), "white")
0213:         elif "07_" in name:
0214:             boxes = [{"Pause", 130, accent), ("Listen", 360, green), ("Understand", 590, gold), ("Unite", 840,
"#DF6C4F")}
0215:             for label, x, c in boxes:
0216:                 rounded_card(d, (x, 320, x + 180, 445), c, outline=dark, radius=25)
0217:                 draw_wrapped(d, label, (x + 10, 350, x + 170, 415), get_font(28, True), "white")
0218:                 if x < 840:
0219:                     d.line((x + 185, 383, x + 225, 383), fill=dark, width=8)
0220:                     d.polygon([(x + 225, 383), (x + 205, 365), (x + 205, 401)], fill=dark)
0221:                     d.text((190, 210), "From conflict to samp", fill=dark, font=get_font(48, True))
0222:         elif "08_" in name:
0223:             d.ellipse((360, 160, 840, 640), fill="white", outline=dark, width=8)
0224:             habits = ["Greet", "Share", "Serve", "Reflect", "Forgive", "Thank"]
0225:             for i, label in enumerate(habits):
0226:                 angle = math.radians(i * 60 - 90)
0227:                 x = 600 + 190 * math.cos(angle)
0228:                 y = 400 + 190 * math.sin(angle)
0229:                 d.ellipse((x - 55, y - 55, x + 55, y + 55), fill=palette[i % 4], outline=dark, width=4)
0230:                 draw_wrapped(d, label, (x - 48, y - 18, x + 48, y + 22), get_font(21, True), "white")
0231:                 draw_wrapped(d, "Daily Values", (500, 355, 700, 435), get_font(34, True), dark)
0232:         elif "09_" in name:
0233:             d.rounded_rectangle((285, 185, 915, 570), radius=35, fill="white", outline=dark, width=7)
0234:             d.rectangle((335, 240, 865, 500), fill="#EAF4F4", outline=dark, width=4)
0235:             for x, c in [(260, accent), (420, green), (780, gold), (940, "#DF6C4F")]:
0236:                 d.ellipse((x - 45, 565, x + 45, 655), fill=c, outline=dark, width=4)
0237:             for label, y in [("Need", 275), ("Plan", 350), ("Serve", 425)]:
0238:                 d.text((385, y), label, fill=dark, font=get_font(32, True))
0239:                 d.line((540, y + 20, 780, y + 20), fill=accent, width=5)
0240:         elif "10_" in name:
0241:             d.rounded_rectangle((335, 165, 865, 575), radius=30, fill="white", outline=dark, width=7)
0242:             d.text((475, 225), "MY PLEDGE", fill=dark, font=get_font(48, True))
0243:             for y in [320, 385, 450]:
0244:                 d.line((430, y, 770, y), fill=accent, width=5)
0245:             for x, y, c in [(260, 245, accent), (940, 245, green), (260, 540, gold), (940, 540, "#DF6C4F")]:
0246:                 d.ellipse((x - 48, y - 48, x + 48, y + 48), fill=c, outline=dark, width=4)
0247:         else:
0248:             d.ellipse((420, 180, 780, 540), fill=accent, outline=dark, width=8)
0249:
0250:         img.save(path, quality=95)
0251:         return path
0252:
0253:
0254: def generate_images():
0255:     paths = []
0256:     for index, (name, title, subtitle) in enumerate(IMAGE_SPECS):
0257:         paths.append(save_image(name, title, subtitle, index))
0258:     return paths
0259:
0260:
0261: def make_styles():
0262:     styles = getSampleStyleSheet()
0263:     styles.add(
0264:         ParagraphStyle(
0265:             name="CoverTitle",
0266:             parent=styles["Title"],
0267:             fontName="Helvetica-Bold",
0268:             fontSize=27,
0269:             leading=33,
0270:             textColor=THEME["navy"],
0271:             alignment=1,
0272:             spaceAfter=16,
0273:         )
0274:     )
0275:     styles.add(
0276:         ParagraphStyle(
0277:             name="PageTitle",
0278:             parent=styles["Heading1"],
0279:             fontName="Helvetica-Bold",
0280:             fontSize=18,
0281:             leading=23,
0282:             textColor=THEME["navy"],
0283:             spaceAfter=8,
0284:         )
0285:     )
0286:     styles.add(
0287:         ParagraphStyle(
0288:             name="SubTitle",
0289:             parent=styles["Heading2"],
0290:             fontName="Helvetica-Bold",
0291:             fontSize=13,
0292:             leading=17,
0293:             textColor=THEME["green"],
0294:             spaceBefore=6,
0295:             spaceAfter=4,
0296:         )
0297:     )
0298:     styles.add(
0299:         ParagraphStyle(
0300:             name="Body",
0301:             parent=styles["BodyText"],
0302:             fontName="Helvetica",
0303:             fontSize=10.5,

```

```

0304:         leading=15,
0305:         textColor=THEME["ink"],
0306:         spaceAfter=6,
0307:     )
0308: )
0309: styles.add(
0310:     ParagraphStyle(
0311:         name="Small",
0312:         parent=styles["BodyText"],
0313:         fontName="Helvetica",
0314:         fontSize=8.5,
0315:         leading=11,
0316:         textColor=THEME["muted"],
0317:     )
0318: )
0319: styles.add(
0320:     ParagraphStyle(
0321:         name="Quote",
0322:         parent=styles["BodyText"],
0323:         fontName="Helvetica-Oblique",
0324:         fontSize=11,
0325:         leading=16,
0326:         leftIndent=12,
0327:         rightIndent=12,
0328:         textColor=THEME["navy"],
0329:         borderColor=THEME["gold"],
0330:         borderWidth=1,
0331:         borderPadding=8,
0332:         backColor=colors.HexColor("#FFF2D0"),
0333:         spaceBefore=6,
0334:         spaceAfter=8,
0335:     )
0336: )
0337: return styles
0338:
0339:
0340: def para(text, style):
0341:     return Paragraph(text, style)
0342:
0343:
0344: def worksheet_lines(count=7, height=0.33 * inch):
0345:     rows = ["" for _ in range(count)]
0346:     table = Table(rows, colWidths=[6.7 * inch], rowHeights=[height] * count)
0347:     table.setStyle(
0348:         TableStyle(
0349:             [
0350:                 ("LINEBELOW", (0, 0), (-1, -1), 0.5, colors.HexColor("#B8C2C8")),
0351:                 ("VALIGN", (0, 0), (-1, -1), "BOTTOM"),
0352:             ]
0353:         )
0354:     )
0355:     return table
0356:
0357:
0358: def box_table(items, col_widths=None, row_height=0.55 * inch):
0359:     data = [[para(item, make_styles()["Body"]) for item in row] for row in items]
0360:     if col_widths is None:
0361:         col_widths = [2.25 * inch] * len(items[0])
0362:     table = Table(data, colWidths=col_widths, rowHeights=[row_height] * len(data))
0363:     table.setStyle(
0364:         TableStyle(
0365:             [
0366:                 ("BOX", (0, 0), (-1, -1), 0.8, THEME["green"]),
0367:                 ("INNERGRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#CBD4D8")),
0368:                 ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#EAF4F4")),
0369:                 ("VALIGN", (0, 0), (-1, -1), "TOP"),
0370:                 ("LEFTPADDING", (0, 0), (-1, -1), 7),
0371:                 ("RIGHTPADDING", (0, 0), (-1, -1), 7),
0372:                 ("TOPPADDING", (0, 0), (-1, -1), 7),
0373:                 ("BOTTOMPADDING", (0, 0), (-1, -1), 7),
0374:             ]
0375:         )
0376:     )
0377:     return table
0378:
0379:
0380: def add_image(story, image_path, caption):
0381:     story.append(Spacer(1, 0.05 * inch))
0382:     story.append(RLImage(str(image_path), width=6.25 * inch, height=3.95 * inch))
0383:     story.append(para(caption, make_styles()["Small"]))
0384:     story.append(Spacer(1, 0.08 * inch))
0385:
0386:
0387: def page_header(canvas, doc):
0388:     canvas.saveState()
0389:     page = canvas.getPageNumber()
0390:     canvas.setFillColor(THEME["navy"])
0391:     canvas.rect(0, PAGE_HEIGHT - 0.38 * inch, PAGE_WIDTH, 0.38 * inch, fill=1, stroke=0)
0392:     canvas.setFillColor(colors.white)
0393:     canvas.setFont("Helvetica-Bold", 8.5)
0394:     canvas.drawString(0.55 * inch, PAGE_HEIGHT - 0.25 * inch, "BAPS-IPDC-WORKBOOK-ITMBU-ED-1")
0395:     canvas.drawRightString(PAGE_WIDTH - 0.55 * inch, PAGE_HEIGHT - 0.25 * inch, "Vaisvik Ekta | Samp | Seva")
0396:     canvas.setFillColor(THEME["muted"])
0397:     canvas.setFont("Helvetica", 8)
0398:     canvas.drawCentredString(PAGE_WIDTH / 2, 0.35 * inch, f"Page {page}")
0399:     canvas.restoreState()
0400:
0401:
0402: def cover_canvas(canvas, doc):
0403:     canvas.saveState()
0404:     canvas.setFillColor(colors.HexColor("#FFF8EA"))
0405:     canvas.rect(0, 0, PAGE_WIDTH, PAGE_HEIGHT, fill=1, stroke=0)
0406:     canvas.setFillColor(THEME["navy"])
0407:     canvas.rect(0, PAGE_HEIGHT - 1.7 * inch, PAGE_WIDTH, 1.7 * inch, fill=1, stroke=0)

```

```

0408: canvas.setFillColor(THEME["saffron"])
0409: canvas.circle(1.0 * inch, PAGE_HEIGHT - 0.86 * inch, 0.28 * inch, fill=1, stroke=0)
0410: canvas.setFillColor(THEME["gold"])
0411: canvas.circle(PAGE_WIDTH - 1.0 * inch, PAGE_HEIGHT - 0.86 * inch, 0.28 * inch, fill=1, stroke=0)
0412: canvas.setFillColor(THEME["navy"])
0413: canvas.setFont("Helvetica", 8)
0414: canvas.drawCentredString(PAGE_WIDTH / 2, 0.35 * inch, "Original educational workbook generated with
Python")
0415: canvas.restoreState()
0416:
0417:
0418: def page(title, body, activities=None, image=None, caption=None, quote=None):
0419:     styles = make_styles()
0420:     story = [para(title, styles["PageTitle"])]
0421:     if image:
0422:         add_image(story, image, caption or "")
0423:     if quote:
0424:         story.append(para(quote, styles["Quote"]))
0425:     for section_title, paragraphs in body:
0426:         story.append(para(section_title, styles["SubTitle"]))
0427:         for text in paragraphs:
0428:             story.append(para(text, styles["Body"]))
0429:     if activities:
0430:         story.append(para("Workbook Activity", styles["SubTitle"]))
0431:         for text in activities:
0432:             story.append(para(text, styles["Body"]))
0433:         story.append(worksheet_lines(activities.count("line") if "line" in activities else 6))
0434:     story.append(PageBreak())
0435:     return story
0436:
0437:
0438: def create_workbook(image_paths):
0439:     styles = make_styles()
0440:     doc = SimpleDocTemplate(
0441:         str(WORKBOOK_PDF),
0442:         pagesize=PAGE_SIZE,
0443:         rightMargin=0.55 * inch,
0444:         leftMargin=0.55 * inch,
0445:         topMargin=0.68 * inch,
0446:         bottomMargin=0.62 * inch,
0447:         title="BAPS-IPDC Workbook 1",
0448:     )
0449:
0450:     story = []
0451:     story.append(Spacer(1, 1.35 * inch))
0452:     story.append(para("BAPS-IPDC-WORKBOOK-ITMBU-ED-1", styles["CoverTitle"]))
0453:     story.append(para("Workbook 1", styles["CoverTitle"]))
0454:     story.append(para("Topic: Vaisvik Ekta, Samp and Seva", styles["CoverTitle"]))
0455:     story.append(Spacer(1, 0.35 * inch))
0456:     story.append(para("A 30-page student workbook for reflection, classroom discussion, and personal
practice.", styles["Body"]))
0457:     story.append(Spacer(1, 0.25 * inch))
0458:     story.append(RImage(str(image_paths[0]), width=6.25 * inch, height=3.95 * inch))
0459:     story.append(Spacer(1, 0.15 * inch))
0460:     story.append(para("Name: _____ Class: _____ Date: _____",
styles["Body"]))
0461:     story.append(PageBreak())
0462:
0463:     pages = [
0464:         (
0465:             "2. How to Use This Workbook",
0466:             [
0467:                 ("Purpose", ["This workbook helps students understand three connected values: Vaisvik Ekta,
Samp, and Seva. Read each short lesson, discuss honestly, and complete the reflection tasks with sincerity."]),
0468:                 ("Learning Method", ["Each page includes a concept, a classroom connection, and an action
step. The aim is not only to know the words, but to practise them in daily life."]),
0469:                 ["Write one personal goal for this workbook.", "line"],
0470:                 None,
0471:                 None,
0472:                 "Values become strong when understanding becomes action.",
0473:             ],
0474:         ),
0475:         (
0476:             "3. Key Words",
0477:             [
0478:                 ("Vaisvik Ekta", ["Vaisvik Ekta means global unity: seeing all people as part of one human
family while respecting different languages, cultures, and traditions."]),
0479:                 ("Samp", ["Samp means harmony, togetherness, and unity in thought, speech, and action. It
asks us to reduce ego and build trust."]),
0480:                 ("Seva", ["Seva means selfless service. It is helping without expecting reward, recognition,
or control."]),
0481:                 ["In your own words, define each value in one sentence.", "line"],
0482:                 None,
0483:                 None,
0484:                 None,
0485:             ],
0486:         ),
0487:         (
0488:             "4. Vaisvik Ekta: One Human Family",
0489:             [
0490:                 ("Concept", ["Global unity begins with the thought that every person has dignity. We may
differ in dress, food, language, or habits, but we share the same need for respect, peace, and belonging."]),
0491:                 ("Classroom Connection", ["A class becomes a small model of the world. When students include
someone who is quiet, new, or different, they practise global unity in a real way."]),
0492:                 ["List three ways your class can show global unity this week.", "line"],
0493:                 image_paths[0],
0494:                 "Image 1: Students around the earth as one human family.",
0495:                 None,
0496:             ],
0497:         ),
0498:         (
0499:             "5. Seeing Beyond Labels",
0500:             [
0501:                 ("Reflection", ["Labels can be useful, but they can also limit how we see people. Vaisvik
Ekta teaches us to look beyond labels and notice character, effort, and inner goodness."]),
0502:                 ("Practice", ["Before judging someone, pause and ask: What do I not know about their story?
This question opens the door to empathy."]),
0503:                 ["Write about a time when you changed your opinion after understanding someone better.",
"line"],
0504:                 None,
0505:                 None,
0506:                 None,
0507:             ],
0508:         ),
0509:     ]

```

```

0500:         ),
0501:         (
0502:             "6. Samp: The Bridge of Harmony",
0503:             [{"Concept", ["Samp is not silence. It is the courage to speak truth with respect and the
maturity to listen without anger."]}],
0504:             ("Four Bridge Stones", ["A strong bridge of samp is built with listening, respect,
forgiveness, and teamwork. If one stone is missing, relationships become weak."])),
0505:             ["Which bridge stone is strongest in you? Which one needs practice?", "line"],
0506:             image_paths[1],
0507:             "Image 2: The bridge stones of samp.",
0508:             None,
0509:         ),
0510:         (
0511:             "7. Harmony in Speech",
0512:             [{"Think Before Speaking", ["Words can repair or damage trust. A samp-filled sentence is
truthful, kind, necessary, and timed well."]}],
0513:             ("Sentence Practice", ["Instead of saying, 'You never help,' say, 'Can we divide this task
more fairly?' The second sentence solves the problem without attacking the person."])),
0514:             ["Rewrite three harsh sentences into samp-filled sentences.", "line"],
0515:             None,
0516:             None,
0517:             None,
0518:         ),
0519:         (
0520:             "8. Seva: Service With a Pure Heart",
0521:             [{"Concept", ["Seva is not only a big project. It can be a small act done with a pure heart:
helping a friend understand homework, cleaning a shared space, or caring for someone who is tired."]}],
0522:             ("Inner Attitude", ["True seva asks: What is needed? How can I help? Can I serve without
making myself the centre?"])),
0523:             ["Describe one small seva you can do at home and one at school.", "line"],
0524:             image_paths[2],
0525:             "Image 3: Seva through food, books, water, and care.",
0526:             "Selfless service turns ordinary moments into meaningful moments.",
0527:         ),
0528:         (
0529:             "9. Why These Three Values Belong Together",
0530:             [{"Connection", ["Vaisvik Ekta gives us the vision: everyone belongs. Samp gives us the
method: live in harmony. Seva gives us the action: help others with humility."]}],
0531:             ("Example", ["If a class wants to serve a community, it first needs unity of vision, harmony
in planning, and selfless effort in execution."])),
0532:             ["Complete this chain: When I believe everyone belongs, I will...", "line"],
0533:             None,
0534:             None,
0535:             None,
0536:         ),
0537:         (
0538:             "10. The Value Tree",
0539:             [{"Roots and Fruits", ["Good actions grow from deep roots. Humility, faith, discipline, and
respect are roots. Unity, harmony, compassion, and service are fruits."]}],
0540:             ("Personal Growth", ["A tree grows slowly but steadily. In the same way, values grow through
repeated practice, not one-time excitement."])),
0541:             ["Draw or write your own value tree: roots, trunk, branches, and fruits.", "line"],
0542:             image_paths[3],
0543:             "Image 4: A value tree with roots and branches.",
0544:             None,
0545:         ),
0546:         (
0547:             "11. Listening Circle",
0548:             [{"Skill", ["A listening circle is a simple method: one person speaks, others listen, and
nobody interrupts. After the speaker finishes, classmates may ask respectful questions."]}],
0549:             ("Why It Works", ["Listening circles reduce misunderstanding because everyone gets space to
be heard. They also train patience and empathy."])),
0550:             ["In a group, discuss: What makes students feel included?", "line"],
0551:             image_paths[4],
0552:             "Image 5: A classroom circle for respectful discussion.",
0553:             None,
0554:         ),
0555:         (
0556:             "12. Empathy Practice",
0557:             [{"Definition", ["Empathy means trying to understand another person's feelings and situation.
It does not mean we agree with everything, but it means we care enough to understand."]}],
0558:             ("Practice Step", ["Use the sentence: 'I can see that you felt ___ because ___.' This helps
another person feel heard."])),
0559:             ["Fill in the empathy sentence for two school situations.", "line"],
0560:             None,
0561:             None,
0562:             None,
0563:         ),
0564:         (
0565:             "13. Seva From Local to Global",
0566:             [{"Scope", ["Seva begins nearby, but its spirit can reach the whole world. A clean classroom,
a helped neighbour, a campus drive, or a relief project all carry the same spirit."]}],
0567:             ("Responsibility", ["Global unity becomes real when people take responsibility for needs
beyond their own comfort."])),
0568:             ["Mark one need at home, on campus, in the city, and in the world.", "line"],
0569:             image_paths[5],
0570:             "Image 6: Seva expanding from home to the world.",
0571:             None,
0572:         ),
0573:         (
0574:             "14. Conflict to Samp",
0575:             [{"Conflict", ["Conflict is natural when people work together. The question is whether
conflict makes us bitter or wiser."]}],
0576:             ("Path", ["Pause before reacting. Listen to the other side. Understand the need behind the
words. Then unite around the shared goal."])),
0577:             ["Choose one conflict and map it through Pause, Listen, Understand, Unite.", "line"],
0578:             image_paths[6],
0579:             "Image 7: A simple path from conflict to samp.",
0580:             None,
0581:         ),
0582:         (
0583:             "15. Ego Check",
0584:             [{"Self-Questioning", ["Many problems grow when ego becomes bigger than the mission. A useful
question is: Am I trying to solve the problem, or only trying to prove I am right?"]]},
0585:             ("Humility", ["Humility does not make a person weak. It makes teamwork possible."])),

```

```

0586:         ["Write three signs that ego is entering a discussion.", "line"],
0587:         None,
0588:         None,
0589:         None,
0590:     ),
0591:     (
0592:         "16. Daily Practice Wheel",
0593:         [{"Small Habits", ["Values become strong through daily habits. Greet people warmly, share
credit, serve quietly, reflect honestly, forgive quickly, and thank sincerely."]],
0594:         ("Challenge", ["Choose one habit for the next seven days. Track your effort each
evening."])],
0595:         ["Select one daily habit and write how you will practise it.", "line"],
0596:         image_paths[7],
0597:         "Image 8: A wheel of daily value habits.",
0598:         None,
0599:     ),
0600:     (
0601:         "17. Case Study: The Group Assignment",
0602:         [{"Situation", ["Four students must submit a project. Two want to lead, one feels ignored, and
one has not completed their part."]],
0603:         ("Apply Values", ["Vaisvik Ekta reminds them that each member matters. Samp helps them speak
calmly. Seva encourages each person to ask, 'What can I contribute now?'"])],
0604:         ["What should the group do in the next 15 minutes?", "line"],
0605:         None,
0606:         None,
0607:         None,
0608:     ),
0609:     (
0610:         "18. Planning a Seva Project",
0611:         [{"Project Steps", ["A seva project needs a real need, a clear plan, shared roles, a
respectful attitude, and reflection after completion."]],
0612:         ("Team Spirit", ["The best project is not the one with the most praise. It is the one that
genuinely helps and makes the team more united."])],
0613:         ["Write a project idea with need, team roles, materials, and timeline.", "line"],
0614:         image_paths[8],
0615:         "Image 9: Students planning a seva project.",
0616:         None,
0617:     ),
0618:     (
0619:         "19. Seva Project Template",
0620:         [{"Use This Template", ["Complete the boxes before starting your project. Keep the plan
simple, realistic, and respectful."]]],
0621:         None,
0622:         None,
0623:         None,
0624:         None,
0625:         None,
0626:     ),
0627:     (
0628:         "20. Reflection After Service",
0629:         [{"After Action", ["Reflection turns activity into learning. After seva, ask what went well,
what could improve, who was helped, and how the team changed."]],
0630:         ("Gratitude", ["Thank the people who allowed you to serve, the team members who helped, and
the mentors who guided you."])],
0631:         ["Write one lesson from service that you want to remember.", "line"],
0632:         None,
0633:         None,
0634:         None,
0635:     ),
0636:     (
0637:         "21. Role Models of Unity and Service",
0638:         [{"Observation", ["A role model is not only famous. A role model can be a parent, teacher,
friend, volunteer, or classmate who quietly lives these values."]],
0639:         ("Study Method", ["Notice the person's action, attitude, and consistency. Then choose one
quality to practise yourself."])],
0640:         ["Name a role model and describe one value you see in them.", "line"],
0641:         None,
0642:         None,
0643:         None,
0644:     ),
0645:     (
0646:         "22. Communication Lab",
0647:         [{"Three Tools", ["Use 'I' statements, ask clarifying questions, and summarise what you heard
before replying. These tools protect samp in difficult conversations."]],
0648:         ("Practice", ["When emotions are high, speak slower and softer. A calm tone can keep the
discussion open."])],
0649:         ["Write a dialogue where two students solve a disagreement peacefully.", "line"],
0650:         None,
0651:         None,
0652:         None,
0653:     ),
0654:     (
0655:         "23. Inclusion Audit",
0656:         [{"Audit", ["An inclusion audit checks whether everyone gets a fair chance to join, speak,
lead, and feel respected."]],
0657:         ("Action", ["Do not wait for excluded students to ask for help. Invite them with warmth and
dignity."])],
0658:         ["Rate your class from 1 to 5 on inclusion. Give one improvement idea.", "line"],
0659:         None,
0660:         None,
0661:         None,
0662:     ),
0663:     (
0664:         "24. My Weekly Tracker",
0665:         [{"Tracker", ["Track your actions for one week. Be honest. The tracker is not for showing off;
it is for becoming aware."]]],
0666:         None,
0667:         None,
0668:         None,
0669:         None,
0670:         None,
0671:     ),
0672:     (
0673:         "25. Quiz: Concepts",
0674:         [{"Answer Briefly", ["Use your own words. Focus on meaning and practical application."]]],

```

```

0675:         ["1. What is Vaisvik Ekta?", "2. What are four bridge stones of Samp?", "3. Why should Seva be
done without expectation?", "4. How can conflict become a chance for growth?", "line"],
0676:         None,
0677:         None,
0678:         None,
0679:     ),
0680:     (
0681:         "26. Scenario Practice",
0682:         ["Choose the Best Response", ["For each scenario, write a response that shows unity, harmony,
and service."]]],
0683:         ["A new student sits alone.", "Your team member made a mistake.", "A campus area is dirty
after an event.", "Two friends are not talking to each other.", "line"],
0684:         None,
0685:         None,
0686:         None,
0687:     ),
0688:     (
0689:         "27. Personal Action Plan",
0690:         ["Plan", ["A good plan is clear enough to start today. Choose one action for each value and
one person who can support you."]]],
0691:         ["My action for Vaisvik Ekta:", "My action for Samp:", "My action for Seva:", "line"],
0692:         None,
0693:         None,
0694:         None,
0695:     ),
0696:     (
0697:         "28. Group Pledge",
0698:         ["Together", ["A pledge becomes powerful when a group supports one another. Read the pledge
aloud and discuss what it will look like in real behaviour."]]],
0699:         ["As a group, write five promises for a more united and service-minded class.", "line"],
0700:         image_paths[9],
0701:         "Image 10: A pledge card for unity, harmony, and service.",
0702:         None,
0703:     ),
0704:     (
0705:         "29. Final Reflection",
0706:         ["Look Back", ["Review your answers from the workbook. Circle one idea that changed your
thinking and one action that changed your behaviour."]],
0707:         ["Look Forward", ["Values education is successful when it continues after the workbook
closes. Choose one practice to continue for the next month."]]],
0708:         ["The biggest lesson I learned is...", "The value I need most now is...", "The action I will
continue is...", "line"],
0709:         None,
0710:         None,
0711:         None,
0712:     ),
0713:     (
0714:         "30. Completion Page",
0715:         ["Student Statement", ["I have studied Vaisvik Ekta, Samp, and Seva. I understand that unity,
harmony, and selfless service must be practised in thoughts, words, and actions."]],
0716:         ["Teacher/Mentor Note", ["Use this page for feedback, encouragement, or a next-step
suggestion."]]],
0717:         ["Student signature: _____", "Teacher/Mentor signature: _____", "Date:
_____", "line"],
0718:         None,
0719:         None,
0720:         "May my thoughts support unity, my words support harmony, and my actions support service.",
0721:     ),
0722: ]
0723:
0724: for index, page_data in enumerate(pages, start=2):
0725:     # Unpack up to 6 items, filling with None if missing
0726:     title = page_data[0]
0727:     body = page_data[1]
0728:     activities = page_data[2]
0729:     image = page_data[3]
0730:     caption = page_data[4]
0731:     quote = page_data[5] if len(page_data) > 5 else None
0732:
0733:     if title.startswith("19."):
0734:         story.extend(page(title, body, None, image, caption, quote)[:1])
0735:         story.append(
0736:             box_table(
0737:                 [
0738:                     ["Need we will serve", "Who will benefit", "Why it matters"],
0739:                     ["", "", ""],
0740:                     ["Team roles", "Materials needed", "Date and place"],
0741:                     ["", "", ""],
0742:                 ],
0743:                 row_height=0.75 * inch,
0744:             )
0745:         )
0746:         story.append(Spacer(1, 0.15 * inch))
0747:         story.append(worksheet_lines(5))
0748:         story.append(PageBreak())
0749:     elif title.startswith("24."):
0750:         story.extend(page(title, body, None, image, caption, quote)[:1])
0751:         story.append(
0752:             box_table(
0753:                 [
0754:                     ["Day", "Ekta action", "Samp action", "Seva action"],
0755:                     ["Mon", "", "", ""],
0756:                     ["Tue", "", "", ""],
0757:                     ["Wed", "", "", ""],
0758:                     ["Thu", "", "", ""],
0759:                     ["Fri", "", "", ""],
0760:                     ["Sat", "", "", ""],
0761:                     ["Sun", "", "", ""],
0762:                 ],
0763:                 col_widths=[0.85 * inch, 1.95 * inch, 1.95 * inch, 1.95 * inch],
0764:                 row_height=0.43 * inch,
0765:             )
0766:         )
0767:         story.append(PageBreak())
0768:     else:

```

```

0769:         story.extend(page(title, body, activities, image, caption, quote))
0770:
0771:     if isinstance(story[-1], PageBreak):
0772:         story.pop()
0773:
0774:     doc.build(story, onFirstPage=cover_canvas, onLaterPages=page_header)
0775:
0776:
0777: def code_line_to_para(line, style):
0778:     safe = (
0779:         line.replace("&", "&amp;")
0780:         .replace("<", "&lt;")
0781:         .replace(">", "&gt;")
0782:         .replace(" ", "&nbsp;")
0783:     )
0784:     return Paragraph(safe, style)
0785:
0786:
0787: def create_code_pdf():
0788:     styles = getSampleStyleSheet()
0789:     styles.add(
0790:         ParagraphStyle(
0791:             name="CodeTitle",
0792:             parent=styles["Title"],
0793:             fontName="Helvetica-Bold",
0794:             fontSize=18,
0795:             textColor=THEME["navy"],
0796:             spaceAfter=10,
0797:         )
0798:     )
0799:     styles.add(
0800:         ParagraphStyle(
0801:             name="CodeBlockTiny",
0802:             parent=styles["Code"],
0803:             fontName="Courier",
0804:             fontSize=6.2,
0805:             leading=7.2,
0806:             leftIndent=0,
0807:             rightIndent=0,
0808:             spaceAfter=0,
0809:         )
0810:     )
0811:     doc = SimpleDocTemplate(
0812:         str(CODE_PDF),
0813:         pagesize=PAGE_SIZE,
0814:         rightMargin=0.42 * inch,
0815:         leftMargin=0.42 * inch,
0816:         topMargin=0.52 * inch,
0817:         bottomMargin=0.45 * inch,
0818:         title="Code Wali PDF",
0819:     )
0820:     story = [
0821:         Paragraph("Code Wali PDF: Workbook Generator", styles["CodeTitle"]),
0822:         Paragraph(
0823:             "This PDF contains the Python code used to generate the workbook PDF and its 10 original
0824:             images.",
0825:             styles["BodyText"],
0826:             Spacer(1, 0.12 * inch),
0827:         )
0828:     ]
0829:     source = Path(__file__).read_text(encoding="utf-8").splitlines()
0830:     for n, line in enumerate(source, start=1):
0831:         numbered = f"{n:04d}: {line}"
0832:         for chunk in wrap(numbered, width=112, replace_whitespace=False, drop_whitespace=False) or [""]:
0833:             story.append(code_line_to_para(chunk, styles["CodeBlockTiny"]))
0834:     doc.build(story, onFirstPage=page_header, onLaterPages=page_header)
0835:
0836: def main():
0837:     image_paths = generate_images()
0838:     create_workbook(image_paths)
0839:     create_code_pdf()
0840:     print(f"Created: {WORKBOOK_PDF}")
0841:     print(f"Created: {CODE_PDF}")
0842:     print(f"Images: {ASSET_DIR}")
0843:
0844:
0845: if __name__ == "__main__":
0846:     main()

```